

Security Best-Practices Checklist

Week 3 — Distilled from the internship work

Author: Maryam Mueen
Intern ID: DHC-1151

Legend. [✓] **Implemented** in `secure-app/`. [] **Out of scope** for this submission, or production-only.

A condensed best-practices checklist for any new web application, distilled from the work done in this internship. Each item is followed by the fix that proves the principle in `secure-app/`.

1 Input Handling

- [✓] **Validate every input at the trust boundary** (server-side, not just the browser).
Fix: `validator.isEmail / isAlphanumeric / isStrongPassword`.
- [✓] **Cap input size** on every endpoint.
Fix: `express.urlencoded(limit: '10kb')`.
- [✓] **Reject early; render the same error message** for “user not found” and “wrong password” so the response does not leak which.

2 Database

- [✓] **Use parameterised queries / prepared statements.** Never string-concatenate user input into SQL.
- [✓] **Principle of least privilege** for the DB account (read-only where possible).
- [✓] **Don't return more columns than the page needs.** Project columns instead of `SELECT *`.

3 Authentication and Sessions

- [✓] **Hash and salt passwords** with `bcrypt`, `scrypt`, or `argon2` (`bcrypt` cost ≥ 12).
- [✓] **Issue signed tokens** (JWT) rather than trusting raw IDs in cookies.
- [✓] **Cookie flags:** `HttpOnly`, `SameSite=Strict` (or `Lax`), `Secure` in production.
- [✓] **Short token lifetime** (≤ 1 hour), refresh on activity.
- [✓] **Rate-limit auth endpoints** to slow brute force and credential stuffing.
- [✓] **Lock-out and back-off** after repeated failures (`express-rate-limit` handles this).

4 Output and Rendering

- [✓] **HTML-escape every variable in templates** (`<%= %>` in EJS, `{{ }}` in Handlebars and React).
- [✓] **Never render user input as raw HTML** unless via a strict sanitiser such as `DOMPurify`.
- [✓] **Set a Content-Security-Policy** that blocks inline scripts and external script sources.

5 HTTP Headers

- [✓] **helmet()** (or equivalent) for the security-header bundle:
 - [✓] **Content-Security-Policy**
 - [✓] **Strict-Transport-Security (HSTS)**
 - [✓] **X-Content-Type-Options: nosniff**
 - [✓] **X-Frame-Options: SAMEORIGIN**
 - [✓] **Referrer-Policy: no-referrer**
- [✓] **Disable X-Powered-By** (`app.disable('x-powered-by')`).
- [] **Use HTTPS for all data transmission.** HSTS is set; production deployment must terminate TLS.

6 Logging and Monitoring

- [✓] **Structured logs (JSON)** with timestamp, level, message, and request context.
- [✓] **Log auth events:** signup, login success, login failure, logout, and token rejection.
- [] **Ship logs to a central store** (Datadog, ELK, CloudWatch) — production must.
- [✓] **Never log secrets, full passwords, or full tokens.**

7 Error Handling

- [✓] **Don't leak stack traces** to clients in production (`NODE_ENV=production`).
- [✓] **Log the full error server-side**, return a generic message to the client.

8 Dependencies and Operations

- [] **Run npm audit regularly** and patch high-severity issues.
- [] **Pin dependency versions** (`package-lock.json` committed).
- [✓] **Secrets in environment variables**, never in source (`.env` + `.gitignore`).
- [✓] `.env` is gitignored in this project; `.env.example` ships as the template.

9 Access Control

- [✓] **Authenticate every non-public endpoint** (middleware like `requireLogin`).
- [] **Authorise** (check role or ownership) on every write — the demo has only one user role.
- [✓] **Don't expose internal IDs or emails to anonymous users** (`/users` is now authenticated).

10 The Three the Brief Asked For

- [✓] **Validate all inputs.**
- [✓] **Use HTTPS for data transmission.** HSTS header set; production TLS termination required.
- [✓] **Hash and salt passwords.**